

Optimal City Navigation Using Markov Decision Processes

1st Aditya Vats
Computer Science 3346A
Western University
London, Canada
avats5@uwo.ca

2nd Evan Probst
Computer Science 3346A
Western University
London, Canada
eprobst2@uwo.ca

3rd Ethan Cawse
Computer Science 3346A
Western University
London, Canada
ecawse@uwo.ca

4th Alan Su
Computer Science 3346A
Western University
London, Canada
msu64@uwo.ca

Abstract—Urban mobility and route optimization are increasingly important challenges. This project investigates the problem of finding an optimal path for a simulated vehicle navigating a city-like grid world with varying traffic conditions and obstacles. We formulate the navigation task as a Markov Decision Process (MDP) in which states represent road intersections, rewards encode traffic penalties, and terminal states correspond to a destination. A value-based planning method is implemented to compute optimal policies, and system behavior is visualized using an interactive grid interface. Multiple road network configurations were evaluated, including maps with misleading shortcuts and varying congestion levels. Results demonstrate that the MDP-derived policy successfully avoids high-cost regions and converges to paths that balance the shortest route selection with traffic efficiency. The findings suggest that value iteration techniques can serve as an effective foundation for autonomous navigation in simplified urban environments and motivate extensions using reinforcement learning or stochastic transition models for more realistic scenarios.

Index Terms—Markov Decision Process (MDP), reinforcement learning, value iteration, urban navigation, optimal policy, grid-based environment

I. INTRODUCTION

Urban navigation and path optimization problems appear in numerous real-world applications ranging from autonomous vehicles to delivery routing and emergency response systems. In these domains, agents must select routes that balance travel efficiency, congestion avoidance, and safety. Artificial intelligence techniques offer scalable solutions for modeling such sequential decision-making problems, where choices made at one location influence future outcomes.

This project investigates the problem of finding an optimal route for an agent navigating a simplified city environment represented as a grid world, which is far removed from actual cities but still useful for problem-solving. States correspond to road intersections, buildings represent impassable barriers, and reward values quantify the desirability of entering a particular region, penalizing congested areas and encouraging movement toward a destination. We formally model this navigation task using a Markov Decision Process (MDP), enabling the use of value-based planning to compute optimal policies.

The motivation behind this work is twofold. First, it provides a controlled testing space to study how MDP algorithms can guide autonomous decision-making. Second, it highlights how reward shaping influences agent behavior, particularly in environments with misleading shortcuts or uneven traffic distributions, mirroring circumstances encountered in the real world.

The primary goals of this project are to (i) design and implement a grid-based navigation engine, (ii) apply MDP value iteration to compute optimal routes, (iii) visualize policy execution across different maps, and (iv) analyze how reward structures and environment layouts affect agent performance. This report serves as documentation as we seek to achieve these goals.

The remainder of this report is organized as follows: Section II reviews relevant work in grid-based planning and reinforcement learning. Section III describes our system design, grid representation, and MDP formulation. Section IV presents experimental results across multiple map configurations. Section V discusses the findings and potential extensions, and Section VI concludes the paper and includes references. Although the abstraction omits elements such as stochastic traffic flow or multi-agent interactions, the simplified representation enables controlled experimentation with planning algorithms.

II. BACKGROUND & RELATED WORK

In this project, urban navigation is represented as a Markov Decision Process (MDP) in which road intersections correspond to states and agent actions reflect directional movements within the grid. Grid-world agents and value-driven planners have been widely used in AI research because they make sequential reasoning interpretable and measurable. Prior studies show that MDP-based approaches can outperform heuristic search in environments where long-term reward trade-offs matter, such as robotic navigation and congestion-aware routing. Reinforcement learning methods such as Q-learning have also been applied to similar domains, demonstrating that agents can learn policies even when transition dynamics are unknown. The deterministic environment assumed in our implementation

means that actions lead directly to intended states, allowing the MDP to capture route planning behavior using a transition model where probabilities collapse to unity for valid moves. By defining reward values that reflect road desirability, the MDP framework allows the system to express route preferences without manually encoding path rules. This formulation enables the computation of globally optimal navigation strategies through policy evaluation, while also serving as a baseline reference for future reinforcement learning extensions that would handle uncertainty or stochasticity in real environments.

To compute an optimal navigation policy, we apply the value iteration algorithm over the constructed MDP. The algorithm initializes state values arbitrarily and iteratively updates them using the Bellman optimality update, propagating expected returns backward from terminal states. In our implementation, this process incorporates traffic-dependent penalties, causing high-congestion areas to accumulate lower long-term value, thereby steering the agent toward more desirable roads. The iterative updates continue until the value function converges below a predefined tolerance threshold, after which an optimal policy is extracted by selecting the highest-valued successor action for each state. Value iteration is particularly well suited to our deterministic environment because it guarantees optimality under known transition dynamics and relative computational simplicity. The resulting policy visibly guides the agent through the grid in our simulator, demonstrating how local reward shaping influences global route preference and how value-based planning can be applied to simplified models of urban mobility.

The Bellman optimality update for value iteration is:

$$V_{k+1}(s) = \max_{a \in A} \left[R(s, a) + \gamma \sum_{s'} P(s'|s, a) V_k(s') \right]$$

Although our implementation does not explicitly write the Bellman equation in symbolic form, it implements it within the value update logic. The function `compute_q_value()` performs Bellman backups by adding local rewards to discounted expected successor values, while the value iteration loop selects the maximum action return, consistent with the optimality operator.

Although we did not explicitly implement a dedicated reward-shaping module, the influence of reward engineering is crucial in how congestion penalties and goal incentives bias policy behavior. By adjusting these embedded reward values rather than hard-coding directional rules. The system implicitly implements reward-shaping, nudging the agent towards desirable routes while discouraging routes with fewer rewards and misleading shortcuts.

III. METHODOLOGY & IMPLEMENTATION

A. Problem Definition & System Overview

The problems that this project addresses are computing an optimal navigation path for an agent moving in a grid-based urban environment. The specifications of this environment, as

mentioned above, include traversable roads, impassable walls in the form of buildings, and a designated goal location. Each grid cell encodes reward values representing the cost incurred for congestion and traffic, encouraging the agent to avoid high-traffic regions. The object is to derive a policy that guides the agent from any valid starting state to the goal while maximizing cumulative rewards.

The implementation consists of two major components: (i) an MDP engine that performs value-based planning and (ii) a visualization system that displays policy execution. Environment configurations are created via CSV files, allowing different grid layouts and traffic profiles to be tested without hard-coding rules and scenarios.

B. Grid Representation & State Encoding

Each map is stored as an $N \times M$ CSV file in which integers represent congestion levels, obstacles, and goal states. The system interprets the following: a. 0 values as drivable road tiles, b. negative values as congestion and traffic penalties, c. a terminal marker to denote the goal

Grid cells correspond to MDP states, indexed using their Cartesian coordinates. Actions consist of four deterministic directional moves: up, down, left, and right. Buildings form absorbing invalid states, and attempted transitions into them return no movement and incur a penalty

C. Markov Decision Process Formulation

Navigation is modeled as an MDP defined by the tuple (S, A, R, P) . States S correspond to coordinate positions. Actions A are movement choices. Rewards R reflect the immediate cost of stepping into a tile. Transition dynamics P assume determinism, collapsing probability distributions to identity mappings for valid moves.

Under this formulation, terminal states correspond to reaching the goal state, and invalid grid entries are excluded from state expansion. The MDP structure enables the computation of optimal policies through dynamic programming rather than manually encoded routing rules.

1) *Stochastic Transition Model*: Although the environment appears grid-deterministic, we implement a simple stochastic transition function. Intended movement succeeds with high probability, but there is a small chance of slipping sideways, which models uncertainty in agent control.

```
# Slip model: with prob 0.8 go intended,
0.1 left, 0.1 right
SLIP_LEFT_RIGHT = {
    "U": ("L", "R"),
    "D": ("R", "L"),
    "L": ("D", "U"),
    "R": ("U", "D"),
}
```

This mechanic introduces probabilistic state transitions into the MDP, encouraging more conservative policies in high-risk regions while still allowing value iteration to converge on optimal behavior.

D. Value Iteration Solver

To compute an optimal policy, the system applies value iteration. State values are initialized arbitrarily, and successive Bellman backups propagate expected cumulative reward towards earlier states. For each state in the model, the planner evaluates all possible actions using a helper routine corresponding to the Bellman update, aggregating immediate reward with discounted successor value. Grid boundaries and obstacles are handled implicitly by the transition function.

The algorithm iterates until convergence, defined as the maximum change between successive value updates falling below a tolerance threshold. This ensures that further updates will not significantly change the derived policy. Once convergence is reached, the optimal policy is extracted by selecting the maximizing action for each state.

1) *Value Backup Computation:* The Bellman optimality update is implemented procedurally by combining local reward with the discounted expected value of successor states:

```
expected_value = (  
    p_forward * V[s_f[0]][s_f[1]] +  
    p_left    * V[s_l[0]][s_l[1]] +  
    p_right   * V[s_r[0]][s_r[1]]  
)  
return R + gamma * expected_value
```

This implementation shows how our planner computes the expected return over slip outcomes and adds it to the immediate reward, matching the Bellman update described earlier in the report.

2) *Policy Extraction Mechanism:* Once the values converge, the system derives an optimal policy by selecting the action with the highest backed-up value at each state. This process scans all admissible actions, evaluates expected returns via the same Bellman-based backup, and records the maximizing action.

```
best_a = None  
best_q = float("-inf")  
for a in ACTIONS:  
    q = compute_q_value(grid, V, i, j, a,  
        gamma, p_forward, p_left, p_right)  
    if q > best_q:  
        best_q = q  
        best_a = a  
policy[i][j] = best_a
```

This snippet illustrates how the planner uses greedy action selection, completing the value iteration loop by converting numerical state estimates into a directional control policy.

E. Reward Modeling and Behavior Sampling

The use of congestion penalties implicitly implements reward shaping. Roads with high traffic density accumulate large negative returns across value backups, discouraging traversal even when the path is geographically shorter. Conversely, the goal receives a positive terminal reward, creating a value gradient that pulls the agent towards it. This balance allows

the system to prioritize reward-aware optimality over minimal path length.

F. Implementation Details

The MDP and planning logic reside inside the file `MDP.py`. The file includes state iteration, Bellman backups, reward retrieval, and policy extraction. Visualization occurs in the file `visualizer.py`. This file renders the grid, overlays policies as directional arrows, and animates agent traversals.

Multiple environment files, such as `grid.csv`, `grid1.csv`, `big_grid.csv`, and `grid_bad_shortcut.csv`, support empirical testing under varied conditions that test our agent across different scenarios. Files such as `car.png`, `road.png`, and `building.png` provide visual grounding within the simulation.

G. Evaluation Plan

Evaluation consists of three components:

- 1) **Grid-based experimentation:** Running value iteration on multiple map topologies allows comparison of how reward structures and geometry influence the resulting routes.
- 2) **Policy inspection through visualization:** The rendered arrows enable qualitative assessment of whether the agent avoids misleading shortcuts or congested regions, thereby reflecting value propagation behavior.
- 3) **Behavioral demonstration:** The agent animation serves to validate that extracted policies lead to goal-directed motion aligned with the computed values.

IV. RESULTS & ANALYSIS

A. Experimental Setup

We tested our value iteration solver in four different CSV map configurations, including `grid.csv` (3x4), `grid1.csv` (3x3), `grid_bad_shortcut.csv` (4x4), and `big_grid.csv` (8x10). The walls were represented as `##` and the goal state as a terminal reward of 10.0 in all experiments. Unless stated otherwise, the discount factor $\gamma = 0.9$ and the convergence tolerance $\theta = 0.01$ were used as consistent parameters as our solver defaults. We also tested a "slip" transition model, where an agent has a 0.8 probability of moving in its desired direction. With a probability of 0.1, it slips to the left, and with a probability of 0.1, it slips to the right. For each map, we report the number of iterations until convergence, the extracted policy's rollout length from the starting cell, and the value of the converged start state $V(s_0)$ as a summary of discounted return.

TABLE I
SUMMARY OF POLICY BEHAVIOR IN MAP CONFIGURATIONS.

Map	Size	Sweeps	Steps	LB	$V(s_0)$
grid	3×4	16	5	3	4.86
grid1	3×3	14	2	2	7.56
grid_bad_shortcut	4×4	18	9	3	1.83
big_grid	8×10	27	16	16	0.97

B. Observed Policy Trends

Across all configurations, the computed policy reached the terminal goal and chose routes that reflect congestion rather than geometric distance. In `grid.csv`, the agent took a 5 step detour vs a 3 step lower bound due to blocked corridors, which shows that the policy adapts to impassable buildings while still prioritizing taking low-penalty roads. In the compact baseline (`grid1.csv`), the agent chose the shortest route to the goal, showing that when the goal is nearby and congestion is mild, the planner acts similarly to shortest path routing algorithms.

The misleading shortcut scenario (`grid_bad_shortcut.csv`) shows the strength of reward-based planning. Despite the goal being only three moves away, the route passes through two -5 cells. The optimal policy instead takes a longer route through eight -0.1 cells, avoiding the higher cost region. In this case, the shorter 3-step path would cost -10 in total state penalties, while the chosen path costs -0.8 in penalties, making the longer path better under the reward model. This phenomenon is increasingly present with the slip transition model. When severe penalties are adjacent to the intended path, lower probability sideways deviations increase the expected cost, showing how the policy routes with a buffer away from risky tiles.

On the map (`big_grid.csv`), the extracted policy takes a clearer route that avoids (-0.1) traffic tiles, and stays primarily on lower-cost roads (-0.01 and -0.2), and still reaches the Manhattan lower bound of 16 steps. This shows that in a mixed cost environment, the planner can keep the route short while avoiding heavily congested areas by accepting the tradeoff of small local penalties for a higher total return.

C. Convergence, Efficiency, and Limitations

Value iteration finished in a smaller number of passes on every map (only 14-27 sweeps in Table I), and it (expectedly) needed more sweeps on larger grids. The start state value $V(s_0)$ is a useful performance indicator, as it shows maps with unavoidable congestion or longer paths (such as `big_grid`) conclude with a lower $V(s_0)$ value despite the same terminal reward. This is because the agent accumulates more penalties along the way, and discounting lowers the total return.

Our evaluation is mostly qualitative, which makes results easier to interpret, but also leaves limitations. Firstly, our visualization follows the greedy extracted policy each step, even though the MDP includes slip probabilities, showing the intended policy but not random slip behavior over multiple runs. Secondly, traffic is fixed, so our approach does not capture real-world time-changing congestion, multiple agent

interactions, or traffic learned from data. Finally, we did not compare our planner against other baseline planners (such as Dijkstra/A*) or other learning based methods, so results should not claim to be better than other routing algorithms, as our results only show that the planner responds to reward costs.

V. CONCLUSIONS & FUTURE WORK

A. Summary of Achievements & Lessons

In this project, we designed and implemented a grid-based navigation system that models urban route selection as a Markov Decision Process. The main achievement is a working planning pipeline that is able to load configurable maps from CSV files, calculate an optimal policy using value iteration with discounted returns, and visually demonstrate the policy and agent path in a grid interface. The planner was able to effectively and consistently reach the terminal goal as well as select routes that reflect reward tradeoffs rather pure distance across multiple grid layouts. Most importantly, our system shows reward-based behavior by avoiding highly negative/congested tiles despite appearing to be the shorter path, while still displaying minimal step counts on larger maps when lower-cost corridors are present.

A key lesson learned was that small changes in rewards can drastically change the preferred route. This is more prevalent with the slip model, as moving close to a high penalty area is riskier, and a small chance of slipping raises the expected cost. We also learned that because of this randomness, the policy usually chooses paths that maintain a buffer from high-cost tiles to stay safer. Finally, we observed that value iteration acts as a baseline for route optimization, as the converged state values reflect expected return, making the policy easy to interpret and debug compared to approaches where it is hard to understand why a particular decision was made.

B. Possible Extensions

Although our system successfully demonstrates value-based planning over grid-structured environments, several extensions would increase realism, scalability, and autonomy. One possible extension would be to replace static value iteration with online reinforcement learning, such as Q-learning, allowing the agent to adapt to changing traffic or incomplete information, mirroring the real-world circumstances that are of concern. Similarly, introducing stochastic traffic models rather than fixed slip probabilities would enable the study of risk-aware planning under uncertainty.

One natural enhancement is benchmarking against heuristic search methods, such as A*, to measure how reward-driven planning compares to shortest path strategies. Beyond learning-based adaptations, analytical comparisons to traditional graph search strategies are another valuable research area. Another promising direction concerns automating reward construction. Our current formulation uses manually assigned congestion penalties, but modern techniques such as inverse reinforcement learning or preference-based modeling could infer reward signals from observed trajectories or driver preferences. This would shift the system from expertly designed behavior

toward data-driven policy shaping, making it more adaptable to complex environments.

Even in its simplified and limited scope, our project addresses core concepts employed in autonomous navigation systems. With refinements and extensions, this system could influence self-driving car planners, fleet routing systems for logistics, municipal congestion modeling, or emergency services response routing. These domains are cost-sensitive, where decision-making occurs over spatial environments.

Collectively, these potential extensions illustrate how our foundational prototype can evolve into a flexible, learning-enabled navigation framework aligned with ongoing challenges in AI planning and autonomous mobility.

VI. INDIVIDUAL CONTRIBUTIONS & PROFESSIONALISM

This project was completed collaboratively by all four team members. Work was distributed across research, implementation, documentation, and testing.

A. *Evan Probst*

Contributed significantly to coding logic, debugging, and early direction-setting discussions within the group. Worked on map design and assisted with issues that arose during implementation.

B. *Alan Su*

Implemented core coding logic alongside Evan, following discussions conducted within the group. Assisted in map design and worked on debugging.

C. *Ethan Cawse*

Took on large parts of the report, such as Results & Analysis, and summarized the group's findings through testing. Communicated closely with the group during the ideation phase.

D. *Aditya Vats*

Authored major sections of the written report, including but not limited to Introduction, Background, and Methodology. Worked alongside the group during the ideation phase. Assisted in polishing documentation and wrote the necessary README file.

E. *Summary*

The team collaborated through a shared messaging group, which served as the primary channel for planning meetings, sharing progress updates, and resolving questions during independent development. Weekly Zoom sessions were held to discuss project direction, assign responsibilities, work through implementation details, and finalize components collaboratively. Meetings were documented either through written notes or stored group messages to maintain transparency. Work was intentionally divided into complementary roles to allow members to progress independently while avoiding duplication of effort: Alan and Evan focused on implementation, coding, dataset preparation, visualization, and video production, while Ethan and Aditya concentrated on research, report writing,

AI methodology articulation, and documentation. Throughout development, all members contributed both individually and collectively, demonstrating effective communication, accountability, and professionalism.

REFERENCES

- [1] Y. Mohsenzadeh, "Search Other Agents", CS3346A Lecture 04, Western University, Fall 2025.
- [2] Y. Mohsenzadeh, "Markov Decision Processes I", CS3346A Lecture 05, Western University, Fall 2025.
- [3] Y. Mohsenzadeh, "Markov Decision Processes II", CS3346A Lecture 06, Western University, Fall 2025.
- [4] Y. Mohsenzadeh, "Reinforcement Learning", CS3346A Lecture 07, Western University, Fall 2025.